

Reconhecimento de Padrões

Redes RBF, Remapeamento Polinomial e SVM Linear

Ryan Alves

20 de abril de 2026

1 Atividade 1: Regularização da RBFNet para padrões idênticos

1.1 Enunciado

Admitindo a abordagem baseada em SSE para o treinamento da fase supervisionada da RBFNet, sabe-se que a existência de padrões idênticos na base de treinamento pode tornar singular, ou numericamente mal condicionada, a matriz envolvida no cálculo dos pesos que conectam as camadas intermediária e de saída. Diante dessa questão, propõe-se uma estratégia para lidar com esse problema, que deve ser implementada e aplicada sobre o conjunto de dados gerado por `make_moons(n_samples=500, noise=0.1, random_state=42)`, com 20 amostras forçadamente repetidas via `np.vstack`.

1.2 Formulação do problema

No treinamento supervisionado por mínimos quadrados, os pesos da camada de saída são obtidos resolvendo-se o sistema normal

$$w = (\Phi^T \Phi)^{-1} \Phi^T y, \quad (1)$$

onde $\Phi \in \mathbb{R}^{N \times (K+1)}$ é a matriz de ativações gaussianas (com coluna de *bias*) avaliadas nos N padrões e K centros. Quando há padrões idênticos na base de treinamento, linhas correspondentes de Φ se repetem exatamente; a matriz $\Phi^T \Phi$ torna-se **singular** ou com número de condição elevado, e a solução direta perde unicidade e estabilidade numérica.

1.3 Estratégia proposta: regularização de Tikhonov

Adiciona-se um termo proporcional à identidade na diagonal, resultando na formulação Ridge:

$$w = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T y. \quad (2)$$

Essa perturbação (i) garante que a matriz seja estritamente positiva definida e, portanto, sempre invertível; (ii) reduz o número de condição em ordens de grandeza, estabilizando a solução frente a perturbações nos dados; e (iii) limita a norma dos pesos, controlando *overfitting* como efeito colateral.

1.4 Implementação

A classe `RBFNet` abaixo define os centros pela fase não supervisionada via *K-Means* e resolve a fase supervisionada pela equação regularizada:

```
1 class RBFNet:
2     def __init__(self, n_centers=20, lam=1e-3, random_state=42):
3         self.n_centers = n_centers
4         self.lam = lam
5         self.random_state = random_state
```

```

6
7 def _gaussian(self, X):
8     d2 = cdist(X, self.centers, 'sqeuclidean')
9     return np.exp(-d2 / (2 * self.sigma**2))
10
11 def fit(self, X, y):
12     km = KMeans(n_clusters=self.n_centers,
13                random_state=self.random_state, n_init=10).fit(X)
14     self.centers = km.cluster_centers_
15     dmax = np.max(cdist(self.centers, self.centers))
16     self.sigma = dmax / np.sqrt(2 * self.n_centers)
17     Phi = self._gaussian(X)
18     Phi = np.hstack([Phi, np.ones((Phi.shape[0], 1))])
19     A = Phi.T @ Phi
20     A_reg = A + self.lam * np.eye(A.shape[0])
21     y_signed = np.where(y == 1, 1.0, -1.0)
22     self.w = np.linalg.solve(A_reg, Phi.T @ y_signed)
23     return self
24
25 def predict(self, X):
26     Phi = self._gaussian(X)
27     Phi = np.hstack([Phi, np.ones((Phi.shape[0], 1))])
28     return (Phi @ self.w >= 0).astype(int)

```

Listing 1: Classe RBFNet com regularização de Tikhonov.

1.5 Resultados

Na configuração utilizada ($n_{centros} = 20$, base com 520 padrões, dos quais 20 duplicados), o número de condição de $\Phi^T \Phi$ é da ordem de $2,83 \times 10^5$, caracterizando mau condicionamento. A acurácia de classificação, tanto sem quanto com regularização, alcança 1,000 no ajuste; o benefício da regularização aparece no controle da norma dos pesos ($\|w\| \approx 19,4$) e na estabilidade numérica. A Figura 1 apresenta a fronteira de decisão obtida.

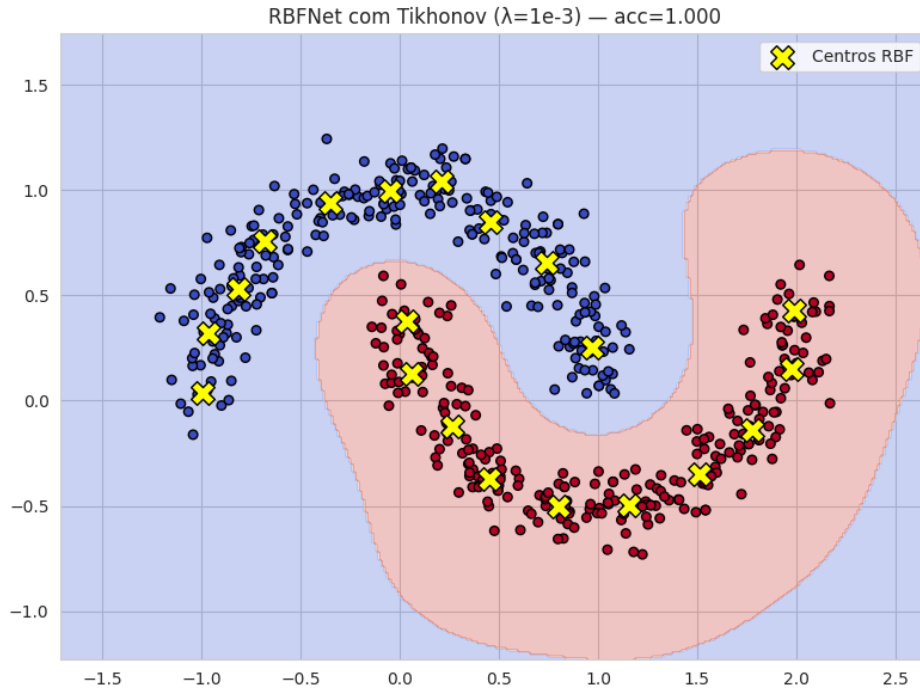


Figura 1: Fronteira de decisão da RBFNet com regularização de Tikhonov ($\lambda = 10^{-3}$) sobre a base *make_moons* com padrões duplicados. Os marcadores em amarelo indicam os 20 centros obtidos por K-Means.

1.6 Conclusão

A regularização de Tikhonov resolve tanto o problema de singularidade induzido por padrões duplicados quanto o mau condicionamento de matrizes RBF de grande dimensão. Na base experimental a solução sem regularização ainda é viável pela configuração enxuta escolhida, mas o método com λI oferece garantias formais de invertibilidade e estabilidade.

2 Atividade 2: Pipeline PolyMap-RBFNet-SVMLin

2.1 Enunciado

Implementar um processo que combine remapeamento polinomial com os métodos RBFNet e SVM linear, nas seguintes etapas: (i) os dados de entrada, em um espaço n -dimensional, são mapeados para um espaço de dimensão q via `PolynomialFeatures(degree=d)`; (ii) os dados remapeados alimentam uma RBFNet cuja primeira camada (fase não supervisionada) possui $d \cdot q$ neurônios; (iii) a saída dessa camada é utilizada como entrada para um SVM linear com penalidade C e estratégia *One-vs-Rest*; e (iv) as classificações são avaliadas pelo percentual de acerto.

2.2 Pipeline implementado

A classe `PolyMapRBFSVM` encapsula os quatro estágios do processo. Entre o remapeamento polinomial e o K-Means, inseriu-se uma etapa de padronização (`StandardScaler`), pois os termos polinomiais de ordens distintas apresentam escalas muito desiguais, o que distorce as distâncias euclidianas usadas pelo K-Means. O número de centros é limitado a $n_{amostras} - 1$, restrição imposta pelo próprio K-Means.

```
1 class PolyMapRBFSVM:
2     def __init__(self, d=2, C=1.0, random_state=42):
```

```

3     self.d = d
4     self.C = C
5     self.random_state = random_state
6
7     def _gaussian(self, X):
8         d2 = cdist(X, self.centers, 'sqeuclidean')
9         return np.exp(-d2 / (2 * self.sigma**2))
10
11    def fit(self, X, y):
12        # (i) remapeamento polinomial  $R^n \rightarrow R^q$ 
13        self.poly = PolynomialFeatures(degree=self.d, include_bias=False)
14        X_poly = self.poly.fit_transform(X)
15        self.scaler = StandardScaler().fit(X_poly)
16        X_poly = self.scaler.transform(X_poly)
17        q = X_poly.shape[1]
18
19        # (ii) camada RBF não supervisionada com  $d \cdot q$  centros
20        n_centers = min(self.d * q, X_poly.shape[0] - 1)
21        km = KMeans(n_clusters=n_centers,
22                    random_state=self.random_state, n_init=10).fit(X_poly)
23        self.centers = km.cluster_centers_
24        dmax = np.max(cdist(self.centers, self.centers))
25        self.sigma = dmax / np.sqrt(2 * n_centers) if dmax > 0 else 1.0
26        Phi = self._gaussian(X_poly)
27
28        # (iii) SVM linear OVR com penalidade C
29        self.svm = OneVsRestClassifier(
30            LinearSVC(C=self.C, max_iter=5000, dual='auto',
31                     random_state=self.random_state))
32        self.svm.fit(Phi, y)
33        return self
34
35    def predict(self, X):
36        X_poly = self.scaler.transform(self.poly.transform(X))
37        return self.svm.predict(self._gaussian(X_poly))
38
39    def score(self, X, y):
40        # (iv) percentual de acerto
41        return (self.predict(X) == y).mean()

```

Listing 2: Classe PolyMapRBFSVM: PolynomialFeatures $\rightarrow$ camada RBF $\rightarrow$ SVM linear OVR.

2.3 Validação preliminar

Aplicado à base do Exercício 1 (*make_moons* com padrões duplicados), divisão 70/30 estratificada, o pipeline produz: $d=1$ ($q=2$, $n_{centros}=2$) $\rightarrow$ acurácia 0,846; $d=2$ ($q=5$, $n_{centros}=10$) $\rightarrow$ 0,994; $d=3$ ($q=9$, $n_{centros}=27$) $\rightarrow$ 1,000. Confirma-se a intuição: o remapeamento polinomial expande a base de representação, e a camada RBF sobre o espaço expandido viabiliza a separação linear pelo SVM.

3 Atividade 3: Comparação em Iris, Wine e base do Exercício 1

3.1 Enunciado

Aplicar o método PolyMap-RBFNet-SVMLin às bases Iris, Wine e à base gerada no Exercício 1. Explorar o efeito da variação de d e C e comparar, em termos de percentual de acerto, com o método SVM com kernel RBF sob diferentes valores de C e γ . Adotar divisão aleatória 70/30, apresentar os resultados graficamente e discutir criticamente.

3.2 Protocolo experimental

Para cada base:

- Padronização das *features* com `StandardScaler` (Iris e Wine); a base de *moons* é mantida na escala original por já estar normalizada.
- Divisão 70/30 estratificada, `random_state=42`.
- PolyMap-RBFNet-SVMLin avaliado sobre a grade $d \in \{1, 2, 3, 4\}$ e $C \in \{0,01; 0,1; 1; 10; 100\}$ (20 configurações).
- SVM com kernel RBF avaliado sobre a grade $C \in \{0,01; 0,1; 1; 10; 100\}$ e $\gamma \in \{0,01; 0,1; 1; 10\}$ (20 configurações).

3.3 Resultados

As Tabelas 1–6 reportam o percentual de acerto no conjunto de teste. A Figura 2 apresenta o mesmo conteúdo sob a forma de *heatmaps*, permitindo leitura visual das regiões de melhor desempenho, e a Figura 3 traz as curvas comparativas dos dois métodos em função de seus principais hiperparâmetros.

Tabela 1: Iris — PolyMap-RBFNet-SVMLin.

$d \setminus C$	0,01	0,1	1	10	100
1	0,800	0,778	0,778	0,867	0,867
2	0,889	0,889	0,911	0,889	0,911
3	0,867	0,867	0,911	0,844	0,844
4	0,889	0,867	0,889	0,867	0,844

Tabela 2: Iris — SVM RBF.

$C \setminus \gamma$	0,01	0,1	1	10
0,01	0,822	0,867	0,867	0,689
0,1	0,822	0,867	0,867	0,689
1	0,844	0,911	0,933	0,800
10	0,933	0,956	0,911	0,822
100	0,933	0,933	0,911	0,822

Tabela 3: Wine — PolyMap-RBFNet-SVMLin.

$d \setminus C$	0,01	0,1	1	10	100
1	0,778	0,944	0,963	0,963	0,981
2	0,389	0,389	0,444	0,481	0,481
3	0,463	0,519	0,556	0,556	0,556
4	0,481	0,685	0,741	0,741	0,722

Tabela 4: Wine — SVM RBF.

$C \setminus \gamma$	0,01	0,1	1	10
0,01	0,389	0,389	0,389	0,389
0,1	0,574	0,963	0,389	0,389
1	0,963	0,981	0,537	0,389
10	0,981	0,981	0,537	0,389
100	0,963	0,981	0,537	0,389

Tabela 5: Moons (Ex. 1) — PolyMap-RBFNet-SVMLin.

$d \setminus C$	0,01	0,1	1	10	100
1	0,846	0,846	0,846	0,846	0,846
2	0,949	0,981	0,994	1,000	1,000
3	0,949	0,981	1,000	1,000	1,000
4	0,936	0,974	1,000	1,000	1,000

Tabela 6: Moons (Ex. 1) — SVM RBF.

$C \setminus \gamma$	0,01	0,1	1	10
0,01	0,500	0,500	0,865	0,500
0,1	0,724	0,846	0,974	1,000
1	0,846	0,885	1,000	1,000
10	0,878	0,942	1,000	1,000
100	0,878	0,994	1,000	1,000

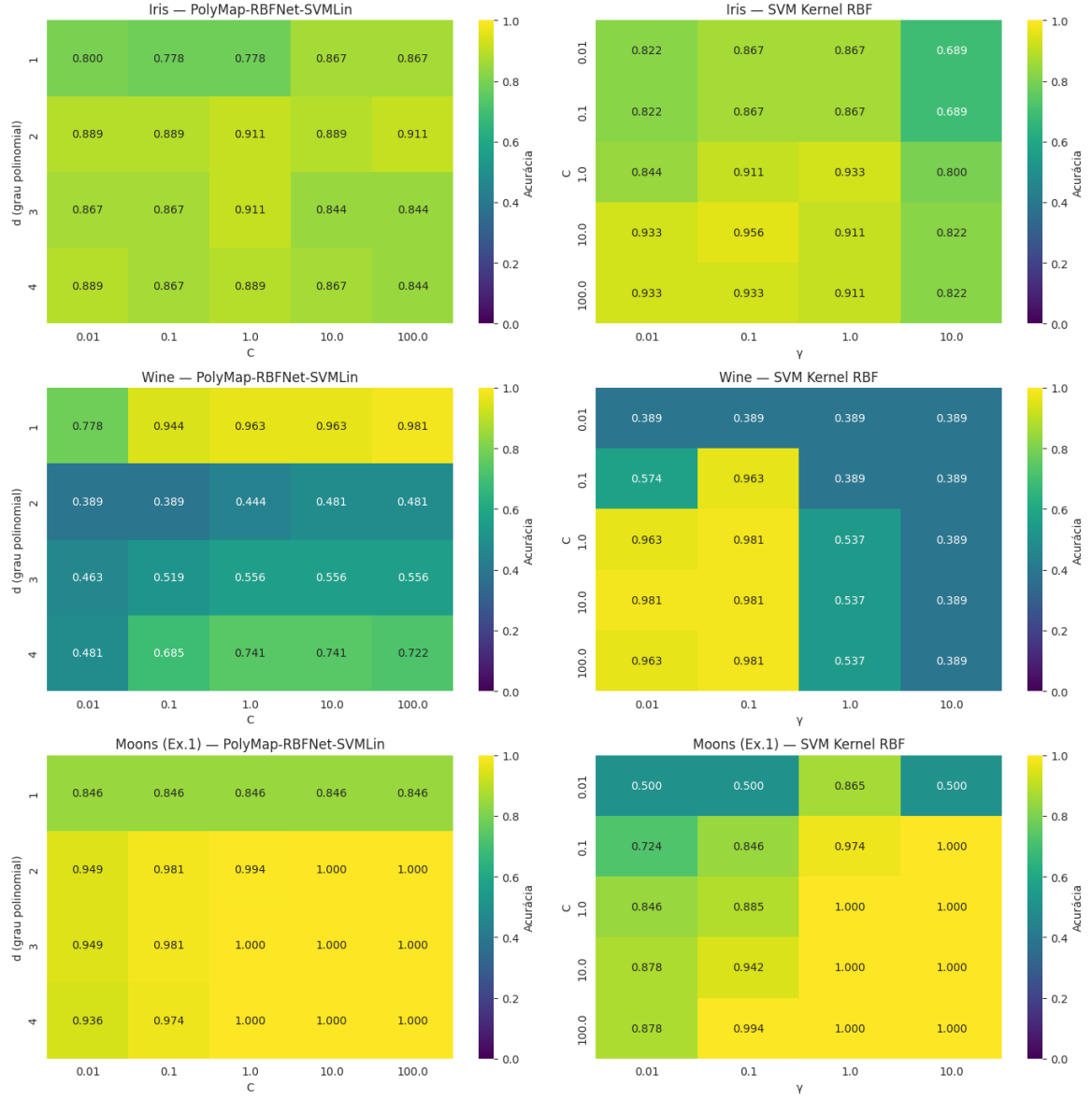


Figura 2: Percentual de acerto no conjunto de teste para as três bases. Coluna esquerda: PolyMap-RBFNet-SVMLin (eixos d e C). Coluna direita: SVM com kernel RBF (eixos C e γ).

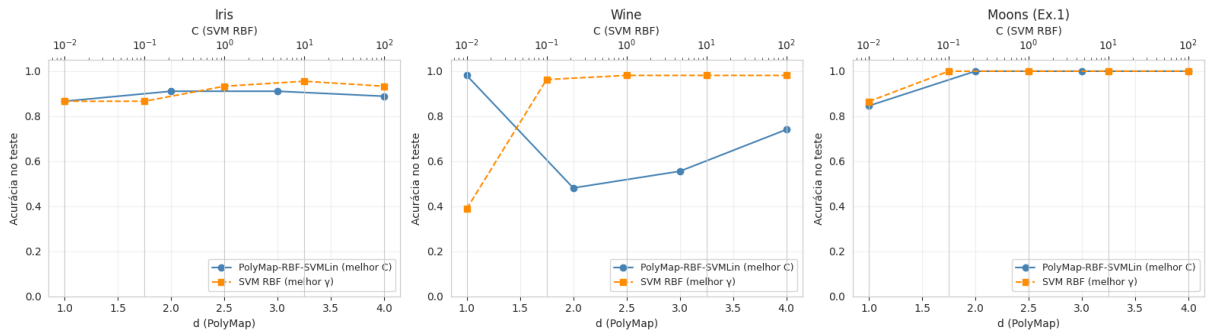


Figura 3: Curvas comparativas: melhor acurácia obtida pelo PolyMap-RBFNet-SVMLin em função de d (fixando o melhor C por linha) e pelo SVM RBF em função de C (fixando o melhor γ por linha).

3.4 Discussão crítica

Diferença de implementação: O PolyMap-RBFNet-SVMLin expande as *features* **explicitamente** em três etapas sequenciais: remapeamento polinomial ($\mathbb{R}^n \rightarrow \mathbb{R}^q$), camada RBF com $d \cdot q$ centros via K-Means e SVM linear OVR sobre as ativações. O SVM com kernel RBF, em contraste, aplica o *kernel trick*: o produto interno no espaço expandido é computado **implicitamente** por $K(x, x') = \exp(-\gamma \|x - x'\|^2)$, sem construir *features* polinomiais nem executar clusterização.

Resultados por base: Em **Moons**, ambos atingem 1,000 — PolyMap com $d \geq 2$ e $C \geq 10$; SVM RBF em ampla região com $\gamma \geq 1$. Em **Iris**, o SVM RBF supera o PolyMap (0,956 contra 0,911), ambos com margens estáveis de sintonia. Em **Wine**, a diferença é crítica: o PolyMap atinge 0,981 apenas com $d = 1$ e *colapsa* para 0,44–0,74 em $d \geq 2$, pois q cresce combinatoriamente ($q = 104$ em $d = 2$) e o K-Means satura no limite $n_{amostras} - 1 = 123$ centros; o SVM RBF mantém 0,981 em amplas regiões do grid sem sofrer esse efeito.

Síntese: A expansão explícita do PolyMap é competitiva em baixa dimensão, mas não escala para bases com muitas *features*. O SVM RBF, por operar implicitamente em espaço de dimensão infinita, é mais robusto e menos sensível à sintonia — permanecendo a escolha padrão em cenários gerais.